
Team Name : GroundTruth

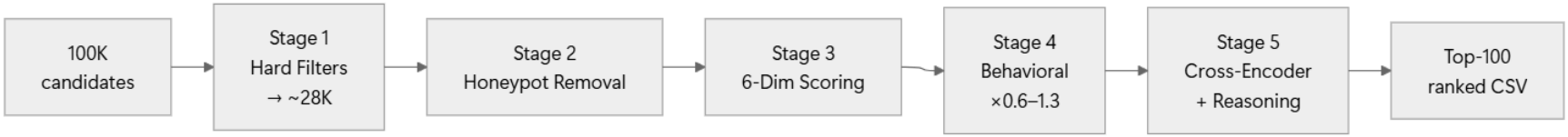
Team Leader Name : Vishnu P S

Problem Statement : Build a local, high-precision candidate discovery & ranking system matching a Senior AI Engineer JD across 100K profiles — seeing through honeypot traps while meeting strict CPU / memory / time limits.

# Solution Overview

**A 5-stage hybrid pipeline that ranks who actually fits the role — not who stuffs keywords.** Most systems either keyword-match (which the stuffers exploit) or call an LLM (which can't run offline in under a minute). We use a staged funnel: cheaply discard non-tech and synthetic profiles first, then spend embedding compute only on real candidates.

## Pipeline




## Why it wins

- ❑ **Skill–career coherence gate** — padded AI skills on a non-tech career drop to 25%. Defeats the JD's central keyword-stuffer trap.
- ❑ **Honeypots = internal impossibility, not description mismatch** — we *rejected* title ↔ description mismatch (it false-flagged 57.8% of the pool) and use only real contradictions.
- ❑ **Semantic over full work history** — BGE embeddings surface plain-language fits who never say "RAG."

**CPU-only · network-off · under 1 minute on a single core · every rank explained.**

# JD Understanding & Candidate Evaluation

**We parse the JD into hard requirements and hidden intent — then score fit on meaning, not keywords.**

**Key requirements extracted from the JD**

- **Role:** Senior AI Engineer (founding team), Series A Indian startup
- **Must-haves:** Python, sentence-transformers, embeddings, vector DBs (Qdrant, Milvus, Pinecone, FAISS), lexical + semantic search, evaluation metrics (NDCG, MAP, MRR, A/B testing)
- **Nice-to-haves:** LLM fine-tuning (LoRA, QLoRA, PEFT), learning-to-rank, distributed systems
- **Hidden intent (the traps):** an all-keywords "Marketing Manager" is not a fit; a Tier-5 who built recommendation systems but never says "RAG" is a fit

**Signals that determine relevance (beyond keywords)**

- **Semantic career meaning** — BGE embeddings over the *coherent* profile fields (title, summary, skills), surfacing plain-language fits
- **Skill credibility** — skills only count when the candidate's actual titles/career support them (coherence gate)
- **Pedigree** — field of study (CS/ML/Stats), institution tier, advanced degrees
- **Logistics & availability** — notice period, ≤50 LPA realism, work mode, engagement/response rate

# Ranking Methodology

**Retrieve → Score → Rank**

- **Retrieve:** hard filters + honeypot exclusion reduce 100K → ~28K real candidates; all scored against precomputed JD/candidate embeddings (cosine similarity)
- **Score (Stage 3):** 6 weighted dimensions — Semantic 0.25 · Career 0.25 · Skills 0.20 · Experience 0.10 · Location 0.10 · Education 0.05
- **Rank (Stage 5):** cross-encoder re-ranks the top-300 for fine-grained ordering

**Models, algorithms & heuristics**

- **Bi-encoder:** BAAI/bge-small-en-v1.5 (384-d) for semantic similarity
- **Cross-encoder:** cross-encoder/ms-marco-MiniLM-L-6-v2 for top-300 re-rank
- **Heuristics:** skill–career coherence gate, internal-impossibility honeypot checks, India-context rules (city, LPA, notice, institution tier)

**How signals combine into the final ranking**

- **Base score** = weighted sum of the 6 dimensions (0–1 each)
- **× Behavioral multiplier** (0.6–1.3) — availability *gates* fit rather than averaging with it
- **Final top-300 order** = blend of **70% bi-encoder + 30% cross-encoder** score
- Result: one transparent composite per candidate, every term traceable to a profile field

# Explainability & Data Validation

**Every rank is traceable to a specific profile field — it is architecturally impossible to cite a skill the candidate doesn't have.**

**How ranking decisions are explained**

- A per-candidate **score card**: all 6 dimensions with the evidence string behind each
- **Rank-aware reasoning** — tone matches tier (specific for top-10, measured for top-30, cautious for 60–100)
- **vs-Naive comparison** and **contrast cards** (why a honeypot was excluded, why a stuffer was demoted)

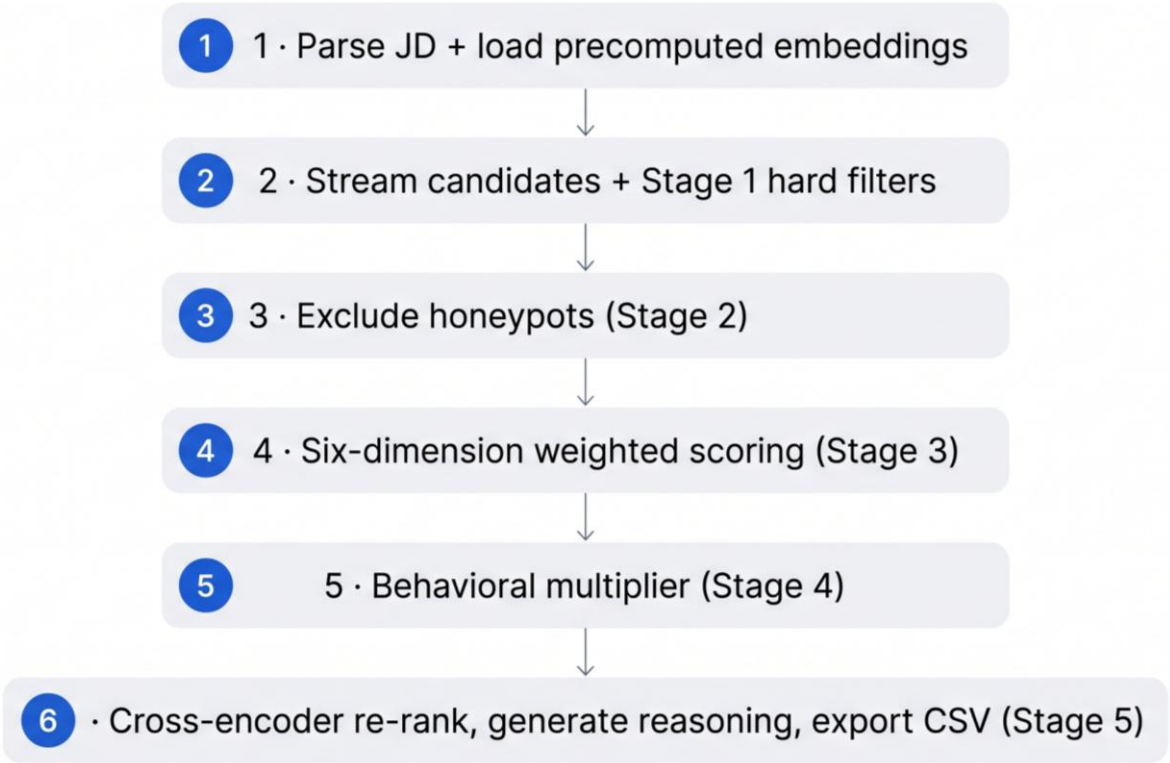
**How we prevent hallucinations / unsupported justifications**

- **Grounded, template-based generation** — no free-form LLM in the scoring or reasoning path
- Every generated claim is **programmatically asserted against actual profile fields, career history, or signals**
- We never quote the dataset's scrambled career descriptions

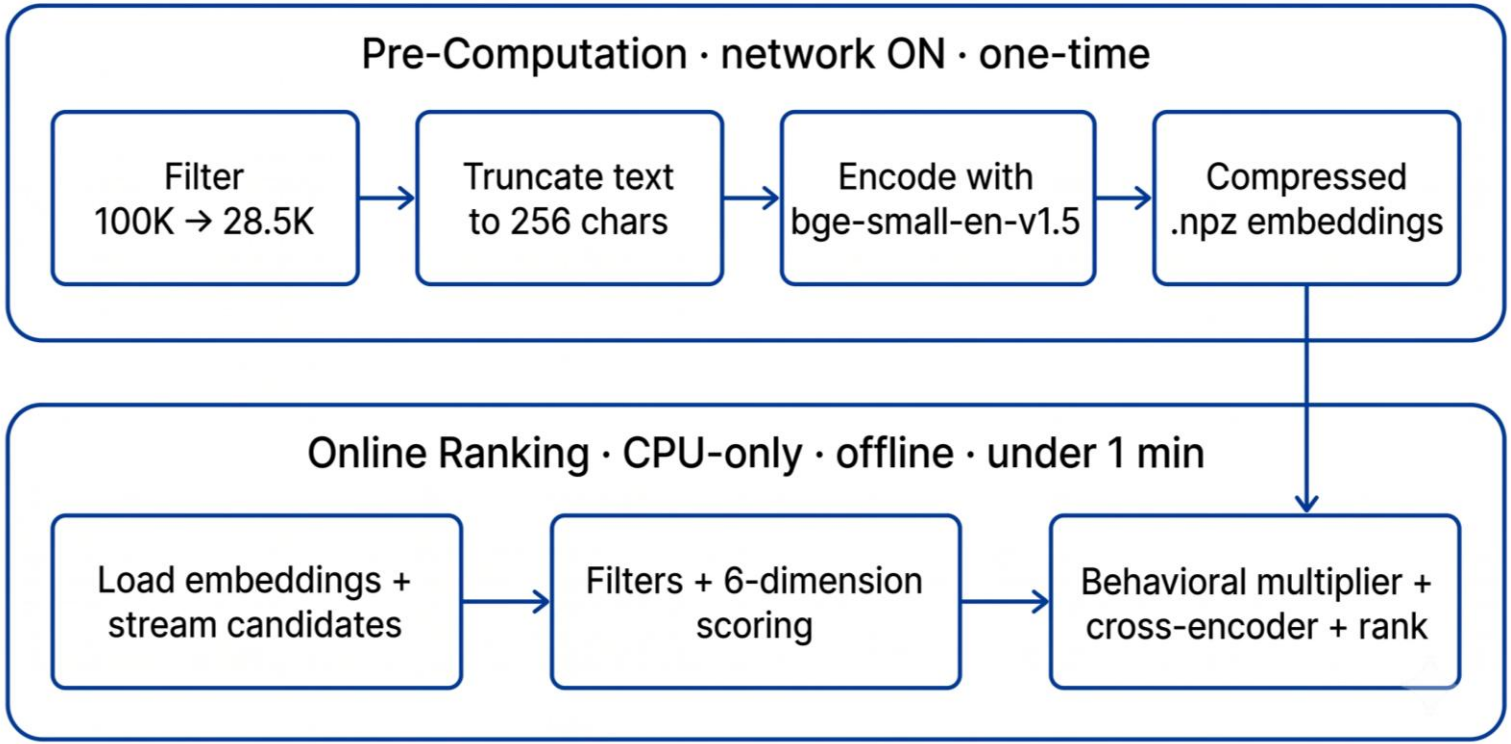
**How we handle inconsistent / low-quality / suspicious profiles**

- **Honeypot detector** on internal impossibilities (expert skill @ 0 months, tenure ≫ experience, overlapping full-time roles) — excluded *before* scoring
- **Skill–career coherence gate** discounts padded skills to 25%
- **Coherence floor** filters incoherent profiles from top ranks

# End-to-End Workflow



# System Architecture



# Results & Performance

**Composite 0.920 on a self-labeled gold set, zero honeypots in the top-10 — ranked in under a minute on a single CPU core.**

Ranking quality (98-candidate gold set)

| Metric | Score |
|---|---|
| **Composite** | **0.920** |
| NDCG@10 | 0.916 |
| NDCG@50 | 0.986 |
| MAP | 0.877 |
| Precision@10 | 0.700 |
| Honeypots in top-10 | **0** |

Composite = 0.50·NDCG@10 + 0.30·NDCG@50 + 0.15·MAP + 0.05·P@10

# Results & Performance

**What drives the ranking (ablation — drop in composite when a signal is removed)**

- ❑ Semantic **−0.079** · Career **−0.067** · Behavioral **−0.033** · Skill–Career Coherence **−0.026** · Cross-Encoder **−0.019** · Experience **−0.008** · Education **−0.003**
- ❑ Confirms semantic + career meaning carry the ranking; every signal contributes positively

**Trap-defeat evidence**

- ❑ Keyword-stuffers with padded AI skills on non-tech careers → discounted to ~25% and demoted out of the top ranks
- ❑ ~113 internally-impossible (synthetic) profiles excluded *before* scoring — 0 surface in results

**Meets the challenge's compute & runtime constraints**

- ❑ **~59s to rank 100K on a single CPU core** — well under the 5-minute limit
- ❑ **CPU-only**, no GPU required
- ❑ **Fully offline** — no network, no hosted LLM/API calls at ranking time
- ❑ **Well within 16 GB RAM** (streamed candidates + compressed embeddings)
- ❑ **Deterministic & reproducible** — same input → same ranking, every run

# Technologies Used

**Every choice serves the constraints: local, CPU-only, offline, explainable.**

- **Sentence-Transformers + PyTorch** — local semantic embeddings (bge-small-en-v1.5) and cross-encoder re-rank; chosen for quality-per-CPU-second (bge-small ≈ 99% of bge-base at 3× speed)
- **NumPy + Pandas** — fast vectorized cosine similarity and data handling
- **Streamlit + Plotly** — interactive tuning dashboard (live weight sliders, radar charts, CSV export)
- **Flask + Next.js / React / Tailwind** — standalone demo API + hosted evaluation dashboard for judges
- **PyYAML** — config & metadata
- **Deliberately no hosted LLM / vector-DB service** — would break the offline + <1 min + reproducibility constraints

## Submission Assets

**Code repository:** https://github.com/Vishnups08/AI-Candidate-Ranking-System
**Live evaluation dashboard:** https://ai-candidate-ranking-system.vercel.app/evaluation
**Interactive sandbox:** https://groundtruth-ai-candidate-ranking-system.streamlit.app/
**One-command reproduce:** python rank.py --candidates ./candidates.jsonl --out ./submission.xlsx (auto-generates submission.xlsx + submission.csv)
**Final output:** submission.xlsx — exactly 100 rows + header, strictly validated

redrob | H2S HACK2SKILL

# *INDIA.RUNS*

Build what next India runs on

# THANK YOU